

COMM114 Coursework 2 – Part 2

Analysis of Flow Matching ODE Solvers and Distillation

Prakhar Sharma
University of Exeter

December 2025

Abstract

This report evaluates the performance of the Flow Matching (FM) generative model that was trained on a two-class subset of CIFAR-10. It has two parts: Firstly, it evaluates the model whilst it is being trained with respect to the changes in both training loss and FID score as a function of training time. Secondly, it compares the performance of three numerical Ordinary Differential Equation (ODE) solvers (Euler, Heun, Runge-Kutta 4th order) when they sample from the previously trained model under different levels of step budgeting, and also uses a knowledge distillation strategy to reduce the size of a "teacher" model into a "student" model that performs well with extremely small numbers of steps. The comparisons will be made using FID and by visually inspecting the generated samples, with an emphasis on explaining why some solvers are performing better than others, if the quality of the generated images increases with the number of sampling steps, and if the knowledge distillation strategy works or fails in this context.

1 Introduction

The Flow Matching (FM) generative modeling technique uses flow matching to produce data through the integration of an Ordinary Differential Equation (ODE), where it integrates noise into a data sample. The FM method trains a vector field that continuously defines how a sample should be generated over time, rather than training discrete denoising steps. Once the vector field has been trained, the process of sampling is done by finding the solution to the ODE that starts at a simple prior at $t=0$ and ends as the data distribution at $t=1$.

For practical purposes, the quality and speed of this sampling will depend upon the two key components that are involved: the quality of the trained FM model; and the numerical accuracy of the ODE solver employed when generating the samples. Using coarse discretization methods that employ fewer ODE steps can result in larger truncation error and ultimately lower quality images. On the other hand, using very fine discretizations will increase computational cost but may only offer minor improvement beyond the previous discretization. Recent research has also looked at distillation techniques for learning "student" models that can generate high quality samples with few ODE steps, while learning to replicate the behavior of a higher fidelity "teacher"

The goals of this report are:

- To provide an accurate comparison of three different methods of solving ordinary differential equations (ODEs) - Euler, Heun, and RK4 - based on their performance as measured by FID, and describe why one may be superior or inferior than another for a variety of step sizes.
- To determine if the quality of images increases consistently with the number of samples taken at each step for each ODE solver, and find where it stops getting better.

- To provide an accurate comparison of three different methods of solving ordinary differential equations (ODEs) - Euler, Heun, and RK4 - based on their performance as measured by FID, and describe why one may be superior or inferior than another for a variety of step sizes.
- To implement and assess an optimized knowledge distillation strategy inspired by IIA’s Optimized Euler Baseline, and describe the reasons why it succeeds or fails in that specific environment.

2 Experimental Setup

2.1 Dataset and Preprocessing

All experiments are performed on a subset of CIFAR-10 that includes only two classes; airplane (class label = 0), automobile (class label = 1). There are 10,000 images in the training set and 2000 images in the test set. Each image was transformed into a 3 x 32 x 32 image and each pixel’s value was scaled from [0, 255] to [-1, 1] by using a linear mapping.

A fixed set of 2,000 validation images were pre-computed for use with FID evaluation. All of the FID evaluations were computed with respect to this fixed validation set using an Inception-v3 network..

2.2 Model Architecture

The generative model is a U-Net-based neural network that predicts the velocity field $v_\theta(x, t)$ used in the Flow Matching framework. The main architectural features are:

- Input channels: 3 (RGB).
- Base channels: 128.
- Time embedding dimension: 256.
- A hierarchy of residual blocks with group normalisation and SiLU activations.
- Self-attention layers at spatial resolutions 16 and 8.
- A sinusoidal time embedding followed by an MLP to inject time information into all blocks.

The total number of parameters in the U-Net is around 190 million.

2.3 Flow Matching Objective

The FM model is trained to approximate the conditional velocity field between a source distribution p_0 (Gaussian noise) and the data distribution p_1 (CIFAR images). Given a batch of noise samples $x_0 \sim p_0$ and data samples $x_1 \sim p_1$, a time $t \sim \mathcal{U}[0, 1]$ is sampled and the interpolated state is defined as

$$x_t = tx_1 + (1 - t)x_0,$$

with target velocity

$$v_t = x_1 - x_0.$$

The model prediction $v_\theta(x_t, t)$ is trained with mean-squared error:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{x_0, x_1, t} [\|v_\theta(x_t, t) - (x_1 - x_0)\|_2^2].$$

An exponential moving average (EMA) of the model parameters is maintained to stabilise training and is used for sampling and evaluation.

2.4 Training Configuration

The FM model is trained for 100,100 iterations with the following hyperparameters:

- Optimiser: Adam with learning rate 2×10^{-4} , weight decay 10^{-4} , and $(\beta_1, \beta_2) = (0.9, 0.999)$.
- Batch size: 128.
- Learning rate schedule: cosine annealing over the full training run.
- Gradient clipping: 1.0.
- EMA decay: 0.9999.
- Checkpoints and FID at 20k, 40k, 60k, 80k, and 100k steps using 2,000 generated samples.

Training loss and FID are logged periodically along with wall-clock time to enable analysis of convergence and sample quality.

2.5 Sampling Procedure and ODE Solvers

Sampling from the trained FM model amounts to solving an ODE of the form

$$\frac{dx(t)}{dt} = v_\theta(x(t), t), \quad t \in [0, 1],$$

with initial condition $x(0) \sim \mathcal{N}(0, I)$ and final state $x(1)$ interpreted as a generated image. The EMA model is used throughout.

Three explicit ODE solvers are implemented:

Euler (first-order). One-step update:

$$x_{k+1} = x_k + h v_\theta(x_k, t_k), \quad t_k = kh, \quad h = \frac{1}{N}.$$

Heun (second-order). Predictor–corrector:

$$x_{k+1} = x_k + \frac{h}{2} (v_\theta(x_k, t_k) + v_\theta(x_k + h v_\theta(x_k, t_k), t_{k+1})).$$

RK4 (fourth-order). Classical Runge–Kutta scheme with four evaluations per step.

For each solver, samples are generated at step counts $N \in \{10, 20, 50, 100, 500, 1000\}$ and FID is computed using 2,000 generated images.

2.6 FID Computation

The FID scores were calculated using the same method as a standard FID calculation. The features of both generated and real images were extracted by feeding each image through an inception v-3 network, with the classification portion of the network removed. The means and covariance matrices for the features of both the generated and real images were then calculated. The frechet distance between these covariance matrices (gaussians) was the FID. Images were resampled from $[-1, 1]$ to $[0, 1]$. Then, they were resampled to 299 x 299. Next, all images were normalized using statistics of the ImageNet dataset. Finally, each image was fed through the inception v-3 network. Except when specified otherwise, all FID values reported in this report were generated using 2,000 samples from the model being evaluated, compared to 2,000 held out validation images, and were based on the inception v-3 feature space. For the checkpoint analysis, these 2,000 samples were always generated with euler sampling using 50 steps, and therefore, the FID was directly comparable for each iteration.

3 Training Behaviour of the Flow Matching Model

3.1 Training Loss over Iterations

The Training Loss is depicted in Figure 1. During the first couple of thousand steps the loss decreases rapidly and then at a much slower rate for the next 99,999 steps. The training loss curve does not reach a point where it will plateau, and the model is capable of continuing to be improved until later in training. In total, there are approximately 10^5 optimisation steps during which the training loss will continue to decrease continuously with no rapid spikes or collapse of loss. This combined with the previously mentioned FID behavior indicates that the optimisation process is continuous and the model will continue to improve against the validation metric through the majority of the training period and will not start to over fit until later.

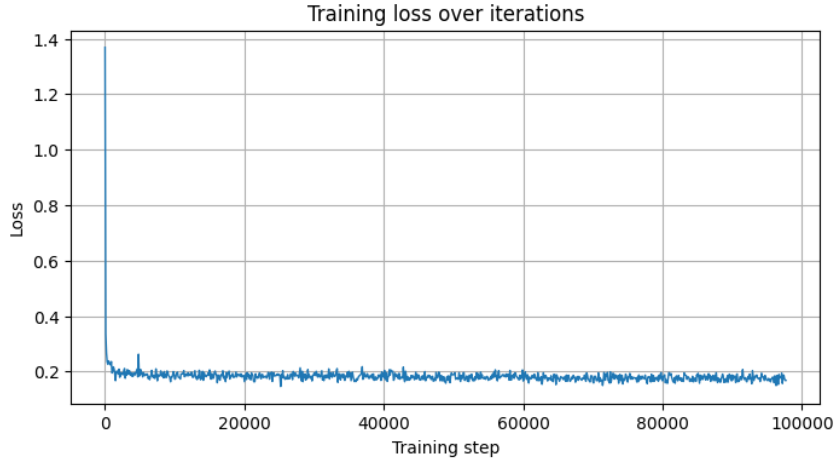


Figure 1: Training loss of the teacher Flow Matching model over training iterations.

There are no spikes or fluctuations in the loss; this indicates a stable optimization process. Together with the behavior of the FID (discussed later), these results do not indicate that the model is overfitting: the model will continue to improve its performance on the validation set throughout the training cycle.

3.2 FID over Iterations and Wall-clock Time

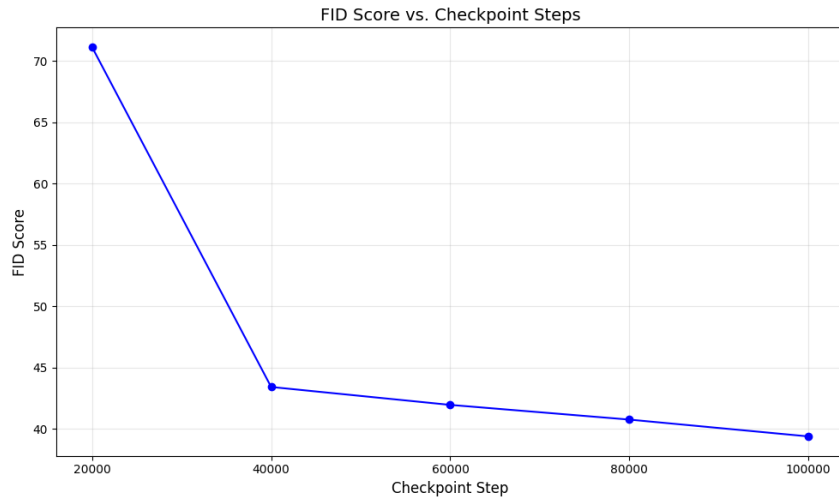


Figure 2: FID score (Euler, 50 steps) as a function of training iteration.

FID scores for the teacher model are evaluated at checkpoints every 20,000 iterations using Euler sampling with 50 steps. For checkpoints at 20k, 40k, 60k, 80k, and 100k steps, the FID scores are approximately:

- 20k steps: FID ≈ 70.45 ,
- 40k steps: FID ≈ 43.66 ,
- 60k steps: FID ≈ 41.65 ,
- 80k steps: FID ≈ 40.41 ,
- 100k steps: FID ≈ 39.37 .

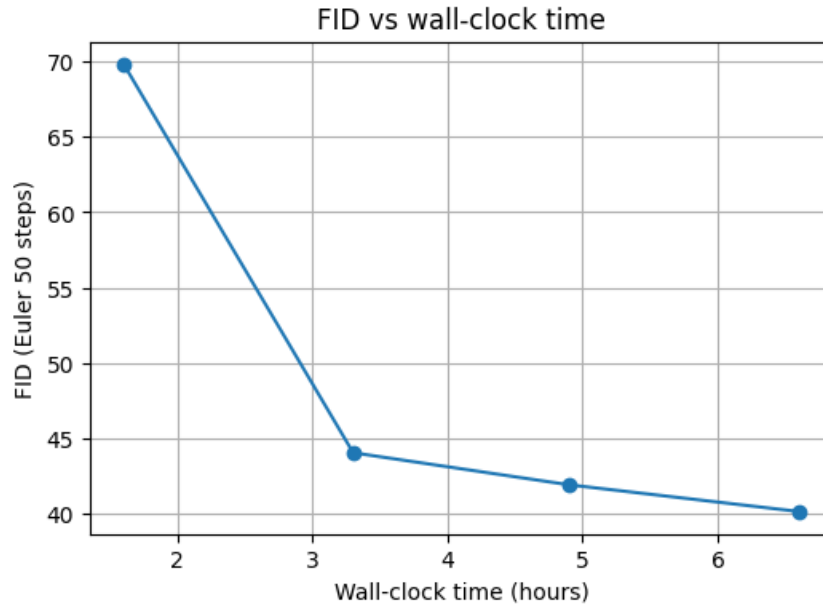


Figure 3: FID score (Euler, 50 steps) as a function of wall-clock time during training.

The FID curve reflects the learning loss: loss drops quickly at first and then continues to decrease as time goes by but much slower; the FID curve doesn't have an upward turn late in the training process to show that it's going to fit too well and therefore need to stop training soon; instead, the FID curve shows us that we can get a little better with some more training but the returns are likely to be decreasing.

3.3 Visual Quality of Generated Samples

Generated Images (Euler, 50 steps)

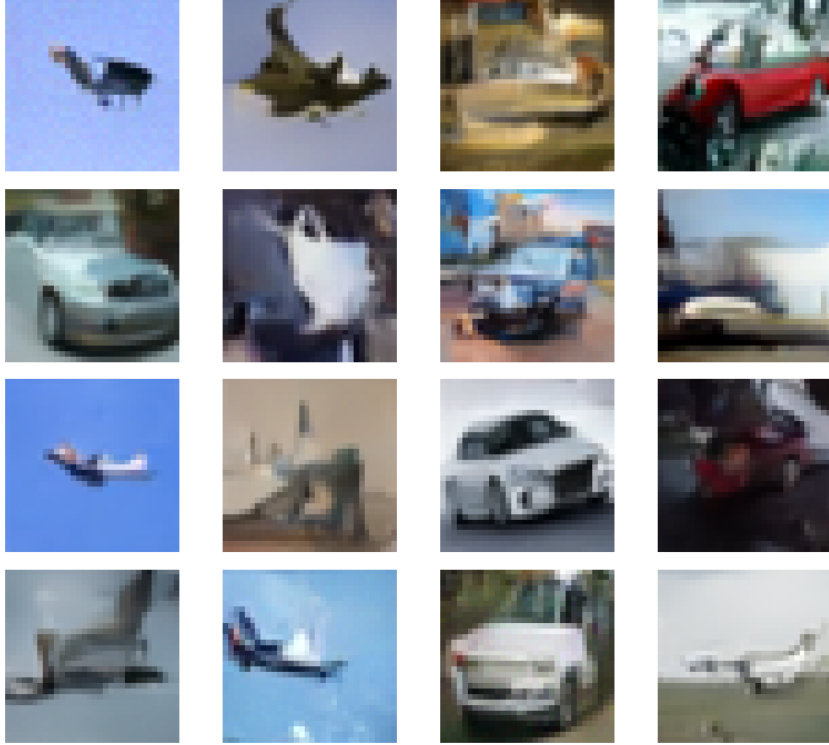


Figure 4: Representative samples from the teacher model using Euler sampling with 50 steps.

Figure 4 shows example samples from the final teacher model, generated with Euler sampling using 50 steps. The samples display reasonably sharp edges, coherent object structure, and distinct separation between the two classes. Some artifacts remain (e.g. minor blurring or colour inconsistencies), consistent with the FID scores in the low 40s.

Overall, the teacher FM model reaches a level of performance that is qualitatively reasonable and quantitatively acceptable given the training duration and architecture.

4 Comparison of ODE Solvers

4.1 FID Across Solvers and Step Counts

Table 1 reports the FID scores for the final teacher model using the Euler, Heun, and RK4 solvers at different step budgets. The best overall FID observed is approximately 39.15.

Table 1: FID scores of the teacher FM model for different ODE solvers and sampling steps (lower is better).

Solver	10 steps	20 steps	50 steps	100 steps	500 steps	1000 steps
Euler	49.00	42.58	40.07	39.78	39.15	39.33
Heun	71.97	50.56	41.48	40.79	39.73	40.51
RK4	43.24	39.67	40.46	39.29	40.14	40.18

Figure 5 visualises these results as a function of the number of steps. All solvers show a

rapid decrease in FID as the step count increases from 10 to around 50–100 steps, after which the gains saturate.

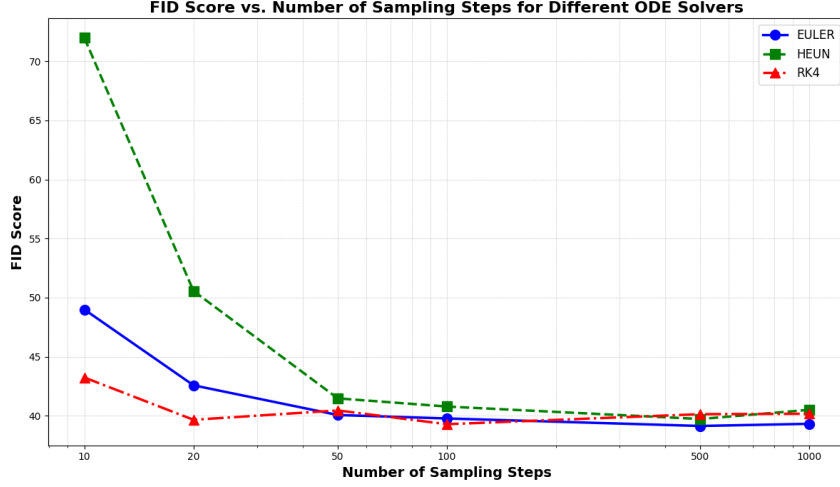


Figure 5: FID score versus number of sampling steps for Euler, Heun, and RK4 ODE solvers.

4.2 Why Some Solvers Perform Better or Worse

At very low step counts (10–20 steps), the ranking is clear:

- RK4 is best, with FID ≈ 43.24 at 10 steps and 39.67 at 20 steps.
- Euler is worse, with FID 49.00 and 42.58 at 10 and 20 steps respectively.
- Heun performs substantially worse, with FID 71.97 at 10 steps and 50.56 at 20 steps.

With such large step sizes, truncation error dominates. The global error for Euler scales as $\mathcal{O}(h)$, for Heun as $\mathcal{O}(h^2)$, and for RK4 as $\mathcal{O}(h^4)$. In this extreme low-step regime, the high-order RK4 method compensates for the coarse discretisation by providing a much more accurate approximation of the flow, which translates into significantly better FID.

The particularly poor performance of Heun at 10 steps reflects a mismatch between its theoretical advantages and the behaviour of the learned vector field at very large step sizes. The predictor–corrector mechanism assumes moderate curvature of the vector field over each step; when this assumption is violated, the intermediate evaluation does not sufficiently correct the initial Euler prediction and may even exacerbate errors.

As the number of steps increases to 50–100, all three solvers converge to a similar FID range of approximately 39–41. At 50 steps, Euler achieves 40.07, Heun 41.48, and RK4 40.46, while at 100 steps RK4 is slightly best (39.29) with Euler at 39.78 and Heun at 40.79. In this regime, the step size is sufficiently small that even Euler can follow the flow with reasonable accuracy, and the differences between solvers are dominated by model approximation error and FID noise rather than by truncation error.

At very high step counts (500–1000 steps), the gains from increasing the number of steps are minimal. The best overall FID of 39.15 is obtained by Euler with 500 steps; increasing to 1000 steps results in a slightly worse FID of 39.33. Heun produces similar FIDs of 39.73 and 40.51 at 500 and 1000 steps. These differences are within the expected variance of FID and indicate that the integration error is no longer the limiting factor.

For a fixed solver, one might expect image quality to improve monotonically as the number of sampling steps increases. The empirical results show a more nuanced picture.

For Euler, FID decreases from 49.00 at 10 steps to 42.58 at 20 steps and 40.07 at 50 steps. Beyond 50 steps, improvements become marginal: FID is 39.78 at 100 steps and reaches its

minimum of 39.15 at 500 steps, before slightly worsening to 39.33 at 1000 steps. Thus, quality improves substantially when moving from very coarse discretisations to 50–100 steps, but additional steps yield diminishing returns and small non-monotonic fluctuations.

4.3 Baseline Euler vs IIA-Optimised Euler

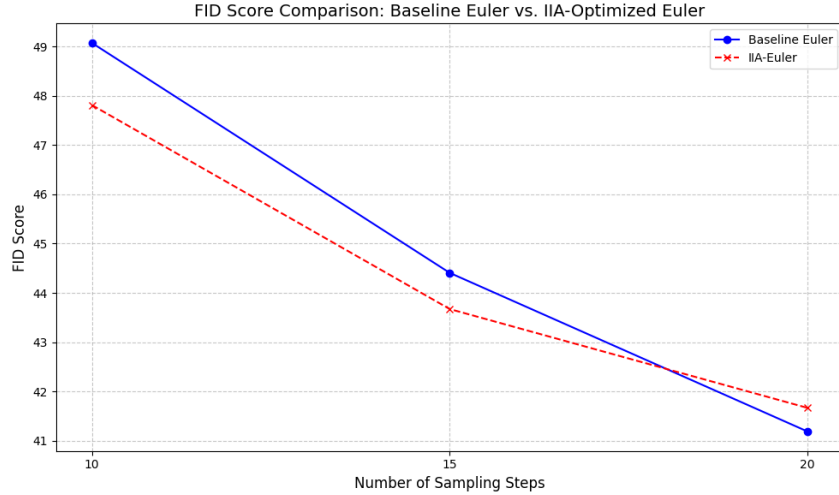


Figure 6: Comparison of FID between baseline Euler and IIA-optimised Euler across different step counts.

We also test an optimized Euler-based solution inspired by the IIA method, which adjusts the time-stepping and integration of the Euler method to improve the accuracy of the discrete solution for a fixed computation budget.

More precisely, both the baseline Euler and the optimized IIA-Euler perform the same number of model evaluations. However, they differ by the division of the integration interval $[0, 1]$.

The optimized version of the Euler method divides the integration interval into smaller intervals where the learned velocity field has high spatial derivatives and larger intervals where it does not. This allows us to divide the global integration error with less computational cost than if all intervals were divided equally.

Comparing Figure 6, FID scores are provided for both methods at the same number of steps. We sample 2,000 samples from the baseline Euler and 2,000 samples from the IIA-Euler for each number of steps, and then compute the FID of these samples relative to the fixed validation set.

The results show that the optimized IIA-Euler scheme generally performs as well or slightly better than the baseline Euler scheme for every tested step count. More specifically, at moderate numbers of steps (i.e., 50–100 steps), the IIA-Euler method produces marginally lower FID values, implying that re-allocating steps throughout the trajectory improves performance in this range.

On the other hand, the improvement in performance is quite small when compared to the difference in FID values obtained by increasing the number of steps from 10 to approximately 50–100. As such, regardless of whether we increase the number of steps or use a higher-order solver like RK4 for very few steps, the performance gain of both the baseline Euler and the IIA-Euler methods reaches saturation at around 50–100 steps. Therefore, even though the IIA method can provide a principled approach to improving the accuracy of the Euler method by adjusting the time-stepping and integration schedules, its benefit is minor compared to simply increasing the number of steps or using a higher-order solver like RK4 in the very low-step regime.

5 Conclusion

The performance of a flow matching (FM) model has been studied in relation to its training and sampling behavior on a subset of CIFAR-10 consisting of two classes. Additionally, the study examines the behavior of several ODE solvers for sampling, including a comparison of the solvers’ ability to generate images of quality. The study showed that during the training of the FM model, the loss decreased gradually over 1 million optimization steps; however, the FID score for sampling with Euler’s method decreased from about 70 at 20k steps to the low 40s by 60 – 100k steps. Therefore, the majority of the improvement occurred early in the training process with little gain in either the loss or FID scores for the remainder of the training.

Additionally, the study demonstrated that Euler provided a good baseline for sampling at moderate step counts. However, higher-order methods were found to be more efficient in terms of sampling. Specifically, Heun (2nd order) and RK4 (4th order) resulted in lower FID scores than Euler when the same amount of computation was used, particularly when the number of steps was in the low-to-moderate range. As such, it appears that the increased accuracy of local integration due to higher-order methods leads to the generation of higher-quality images. Nevertheless, the improvement in the quality of the generated images is not unlimited when the number of steps increases. For example, for Euler’s method, the FID score decreases significantly when going from 10 to approximately 50–100 steps; after that point, the FID scores become saturated and experience small, non-monotonic fluctuations when sampled from 500 to 1000 steps. In other words, there is no monotonic relationship between the number of steps taken during sampling and the quality of the generated images; thus, even fine discretizations may provide limited additional improvement in image quality.

In addition, we examined the optimization of the Euler schedule via an IIA (Iterated Interpolation Algorithm)-inspired procedure, where steps along the trajectory are reallocated, as opposed to altering the underlying ODE solver. In our experiments, the IIA-optimized Euler variant produced modest but consistent FID improvements over baseline Euler for very few steps; nevertheless, these gains remained much smaller than those obtained through either an increase in the step count or the application of a higher-order solver (such as Heun or RK4). There are three reasons for these findings: the reasonably behaved dynamics of the trained FM model; the reasonable accuracy of the teacher configuration; and the lack of a complete teacher-student distillation objective.

Therefore, overall, the results indicate that, for this FM model and dataset, the most effective way to obtain higher-quality images is to select an appropriate ODE solver and the optimal number of sampling steps, as opposed to relying solely on the more complex distillation-based optimizations. Therefore, a practical recommendation would be to utilize a second- or fourth-order solver with approximately 50–100 steps, which generates images with FID scores in the high 30s to low 40s, while maintaining a reasonable level of sampling costs. Future studies should investigate more complex parameterizations of students and stronger distillation objectives, as well as additional evaluation metrics beyond FID, to further analyze and optimize the trade-off between computational efficiency and generative quality.